PROMISE+ Requirements Classification

Phase 3 Final Report: Baseline Development, Model Selection and Independent Evaluation

Study: Project-disjoint 12-class software requirements classification
Phase: 3 - Classical machine-learning model development
Report date: 3 November 2026
Status: Phase 3 concluded; advanced modeling proposed for Phase 4

Executive summary

Phase 3 established and evaluated reproducible classical machine-learning baselines for the PROMISE+ requirements classification dataset. Five baseline configurations were evaluated: a majority-class classifier and TF-IDF with Logistic Regression, Linear Support Vector Machine (SVM), Multinomial Naive Bayes, and Random Forest. The strongest initial model, Linear SVM, achieved a validation macro-F1 of **0.3570**. Subsequent, predefined 24-configuration searches for Linear SVM and Logistic Regression improved their best validation macro-F1 scores to **0.3766** and **0.3894**, respectively.

The selected model was TF-IDF with class-weighted Logistic Regression (`C=10.0`, word n-grams `(1,2)`, `min_df=1`). It was selected according to validation macro-F1 and then evaluated without retraining on the frozen 550-record project-disjoint test partition. Its **final test accuracy was 0.6800 (374/550 correct)**, **macro-F1 was 0.3755**, and **balanced accuracy was 0.3465**. Stronger performance on the majority Functional (`F`) class contrasted with zero test recall for `L` and `SC`. These results demonstrate a functioning baseline and expose important limitations in minority-class recognition and cross-project generalization.

Interpretation: Results are evidence for a baseline under the specified split, not a claim of state-of-the-art performance. Phase 4 will investigate stronger models without using Phase 3 test outcomes for hyperparameter selection.

1. Research objective and scope

The primary task was supervised **12-class classification** of natural-language software requirements. Models received requirement text and predicted one label in the original PROMISE+ label set: `F`, `A`, `L`, `LF`, `MN`, `O`, `PE`, `SC`, `SE`, `US`, `FT`, and `PO`. Phase 3 did **not** evaluate a separate binary Functional/Non-Functional classifier, inconsistency detector, or requirement-quality model. These would require distinct experiments and reporting.

The objectives were to establish conventional baselines, identify a validation-selected classifier, independently measure test performance, and characterize difficult requirement classes and project-level variation.

2. Dataset and leakage-controlled evaluation

Property	Value
Total requirements	3,677

Property	Value
Total projects	88
Classes	12
Training requirements	2,574 (70.00%)
Validation requirements	553 (15.04%)
Test requirements	550 (14.96%)
Training / validation / test projects	50 / 19 / 19
Split selection	Phase 2 Candidate 2, seed 42

The dataset was repaired and inspected during Phase 1; Phase 2 normalized text, prepared labels, and selected a fixed three-way split. Project-group isolation and normalized-text duplicate isolation were enforced across partitions. The split was selected using predefined isolation and class-balance criteria rather than downstream model scores. All 12 classes appeared in all three partitions. The raw data were not randomly mixed across projects for Phase 3.

The final evaluation script verified SHA-256 checksums for all split files, row counts, label coverage, and the absence of cross-partition overlap for project identifiers, normalized requirement texts, and record identifiers. **All integrity checks passed.** The final trained model was loaded from the previously saved artifact, without fitting on the test split.

Reproducibility identifiers

- Train split SHA-256: `b70937f65cbef65aaaad630267c2d81cb3c6f88eba45e38b2504e7871c3e9f69`
- Validation split SHA-256: `7613b615adce2d653fa06f181770295778b55d250e7ca00431746c2c4be84e49`
- Test split SHA-256: `014411167392db6405c9ff61f7cde0aed8629134653cb4714441a68a608fd68b`
- Selected model SHA-256: `b0a5b0336024fda54304f7a4a0ac060da3c2eff88ab6c19e5cf5a1bedb28cfcb`
- Random seed: `42`; reported scikit-learn installation: `1.9.1`.

3. Baseline experimental setup

The text feature extractor was `TfidfVectorizer` with lowercasing, Unicode accent normalization, word unigrams/bigrams, `min_df=2`, `max_df=0.95`, and sublinear term frequency. TF-IDF was fitted **only on the training partition** inside a scikit-learn pipeline. The initial Logistic Regression, Linear SVM, and Random Forest models used balanced class weights. Multinomial Naive Bayes used `alpha=1.0` with learned training-set class priors; the majority-class baseline predicted the most common training label.

Metrics were calculated on the **553-record validation split**. The primary model-selection metric was **macro-F1**, which assigns equal weight to each class when averaging class F1 values. Accuracy, balanced accuracy, macro-precision, and weighted-F1 were reported as complementary indicators. No test examples were used to select baseline configurations.

4. Baseline results: validation partition

Configuration	Accuracy	Balanced accuracy	Macro-F1	Weighted-F1
Majority class	0.6148	0.0833	0.0635	0.4682
TF-IDF + Logistic Regression	0.5642	0.4121	0.3435	0.5781

Configuration	Accuracy	Balanced accuracy	Macro-F1	Weighted-F1
TF-IDF + Linear SVM	**0.6076**	**0.4169**	**0.3570**	**0.6078**
TF-IDF + Random Forest	0.5208	0.3061	0.2640	0.5185
TF-IDF + Multinomial Naive Bayes	0.6166	0.0867	0.0699	0.4722

The Naive Bayes accuracy was the largest among the initial configurations, but its macro-F1 was low. A direct prediction audit found **552 of 553 validation predictions were** `F`, with only one `PE` prediction; ten classes were never predicted. Thus accuracy alone concealed poor multiclass performance. Linear SVM was the strongest initial model according to the predefined macro-F1 criterion.

5. Hyperparameter tuning and validation selection

Two **separate 24-configuration grid searches** were run, one each for Linear SVM and Logistic Regression. Both varied `C` in `{0.1, 1.0, 10.0}`, `class_weight` in `{None, balanced}`, n-gram range in `{(1,1), (1,2)}`, and `min_df` in `{1, 2}`. All configurations were trained on the training partition and ranked on validation macro-F1. The test partition remained untouched during selection.

Model	Baseline val. macro-F1	Tuned val. macro-F1	Tuned val. accuracy	Chosen C	Weight	N-grams	min_df
Linear SVM	0.3570	0.3766	0.6564	1.0	balanced	(1, 2)	1
Logistic Regression	**0.3435**	**0.3894**	**0.6691**	**10.0**	**balanced**	**(1, 2)**	**1**

The tuned Logistic Regression achieved a validation balanced accuracy of **0.4131**, macro-precision **0.4573**, and weighted-F1 **0.6505**. The tuned SVM had validation balanced accuracy **0.4129** and weighted-F1 **0.6423**. Logistic Regression led tuned SVM by about **0.0127 macro-F1** and was selected **before** final test evaluation. Searches on a single validation partition can favor configurations adapted to that partition; their scores should be interpreted as development results.

6. Validation error analysis

The tuned Logistic Regression improved class F1 over tuned SVM for seven labels, performed worse for three, and tied for two. The classifiers disagreed on only **43 of 553** validation records: both were correct on 353, only SVM was correct on 10, only Logistic Regression was correct on 17, and both were incorrect on 173.

The most serious validation failure was `PO`. All **24** validation `PO` requirements belonged to **Project 52**, and none were correctly classified by either tuned model. Logistic Regression predicted these 24 cases as `F` (12), `PE` (4), `SE` (3), `O` (2), `L` (2), or `US` (1). Of the 42 training `PO` requirements, **34 (80.95%) came from Project 59** and the remaining eight from five other projects. Project 52 had **26 validation requirements and zero correct classifications overall**. Project-level accuracy varied from zero in some projects to over 90% in others.

These observations are consistent with a potential mismatch in vocabulary, contexts, and class composition between projects, but do not independently prove the cause. Some PO examples contain language about transmission times, data capacity, and software testing that could resemble other requirement categories. They should **not** be relabeled from model predictions without consulting authoritative annotation guidelines and performing manual review.

7. Frozen final test evaluation

The selected, already-trained Logistic Regression pipeline was evaluated on **550 requirements from 19 held-out projects**. The model and split checksums matched their recorded values. No retraining or hyperparameter optimization occurred during final evaluation.

Metric	Validation (selection)	Final test	Test minus validation
Accuracy	0.6691	**0.6800**	+0.0109
Balanced accuracy	0.4131	**0.3465**	-0.0666
Macro-precision	0.4573	**0.5291**	+0.0718
Macro-F1	0.3894	**0.3755**	-0.0139
Weighted-F1	0.6505	**0.6378**	-0.0127

The final test classifier made **374 correct predictions out of 550**. Overall accuracy increased slightly compared with validation, while macro-F1 and balanced accuracy declined. The fall in balanced accuracy indicates weaker average class recall on the final test projects, despite higher aggregate accuracy.

7.1 Per-class independent test metrics

Class	Precision	Recall	F1	Support
F	0.7644	0.9104	0.8311	335
A	0.7273	0.7273	0.7273	11
L	0.0000	0.0000	0.0000	35
LF	0.6667	0.2000	0.3077	10
MN	0.4000	0.1818	0.2500	11
O	0.2800	0.3333	0.3043	21
PE	0.6111	0.4400	0.5116	25
SC	0.0000	0.0000	0.0000	9
SE	0.5000	0.6857	0.5783	35
US	0.4000	0.2791	0.3288	43
FT	1.0000	0.2000	0.3333	5
PO	1.0000	0.2000	0.3333	10

The majority F class had **91.04% recall** and **0.8311 F1**. By contrast, L and SC had **zero recall**. Precision of 1.0 for FT and P0 should not be interpreted as excellent generalization: each had only **20% recall**, with very small test supports (5 and 10). P0 performance was different from validation (20% versus 0% recall), further illustrating variability between held-out projects.

7.2 Project-level test variation

The final test split contained 19 projects. Examples illustrate the variability:

Project ID	Test records	Correct	Accuracy
57	113	103	0.9115
73	119	81	0.6807
3	79	61	0.7722
66	60	16	0.2667
2	40	25	0.6250
89	38	23	0.6053
35	8	2	0.2500

Projects with one or two records sometimes had 0% or 100% accuracy; those percentages are unstable because of very small denominators. The spread among larger projects, including Project 57 versus Project 66, still provides evidence of substantial heterogeneity in generalization performance.

8. Integrity, reproducibility and saved outputs

The Phase 3 workspace contains individual scripts for baseline training, comparisons, hyperparameter searches, validation error analysis, project-level analysis, and final independent evaluation. Saved outputs include fitted pipeline `.joblib` artifacts, CSV metric summaries, JSON classification reports, class confusion matrices, prediction tables, project performance tables, and comparison figures.

The final selected pipeline is `outputs/models/07_tuned_logistic_regression.joblib`. The independent evaluation report is `outputs/reports/11_final_test_evaluation.json`, accompanied by `outputs/metrics/11_final_test_metrics.csv`, `outputs/tables/11_test_per_class_metrics.csv`, and `outputs/tables/11_test_project_performance.csv`.

The Phase 2 manifest recorded **four unresolved annotation-conflict groups**, which remain pending manual investigation. Their presence should be acknowledged in downstream analyses. The saved output files and SHA-256 identifiers support reproducibility, but results may still depend on software/hardware versions and characteristics of the project split. Binary `joblib` files should only be loaded from trusted sources.

9. Limitations

1. **Class imbalance:** `F` dominates the dataset and many minority classes have small training or evaluation supports. Accuracy can overstate usefulness across the label set.
2. **Unseen-project variability:** The split intentionally isolates projects; terminology and label proportions differ between projects. The model's success on some projects does not guarantee comparable performance on others.
3. **Single fixed split:** This is one particular project-disjoint evaluation, not a confidence interval over many possible held-out projects. Model ranking may change under other project compositions.
4. **Limited representations:** Classical word-level TF-IDF and conventional classifiers may miss contextual, semantic, or domain-specific relations in requirements.
5. **Validation adaptation:** Two 24-configuration searches used the same validation set. The chosen score may reflect some adaptation to that split.
6. **Annotation uncertainty:** Four conflict groups remain unresolved, and label-definition differences may complicate error interpretation. No labels were modified in Phase 3.

7. **Previously opened test split:** The Phase 3 test has been evaluated and analyzed. Its scores and class errors are now known, so it must **not** serve as an untouched selection set for later Phase 4 experiments.

10. Conclusion and transition to Phase 4

Phase 3 successfully established five reproducible baseline configurations, performed 48 validation-only tuning experiments across the two leading linear models, identified difficult classes and project-specific errors, and independently evaluated a frozen selected classifier. The final model achieved **68.00% accuracy, 37.55% macro-F1, and 34.65% balanced accuracy** on the project-disjoint test split. Although adequate as a methodological baseline, its low average class recall and zero recall for two test classes show that substantially stronger generalization is needed.

Phase 4 recommendation: Introduce project-aware development evaluation, character-level and combined feature representations, stronger contextual language models, and carefully justified imbalance strategies. Use only training/development data for these experiments and plan a **new independent evaluation dataset or otherwise genuinely untouched benchmark** for subsequent performance claims. Retain the Phase 3 test result as a transparent historical baseline rather than repeatedly selecting new models against its observed scores.

Source and provenance note

This report is based on the **experiment outputs transcribed in the current thesis workflow**: Phase 2 manifest, Phase 3 terminal results from scripts `01` through `11`, and associated per-class and project-level analyses. It does not claim independent access to or re-execution of the user's local code or datasets. No external literature comparisons are asserted here; literature-based discussion and citations should be added when integrating the report into the full thesis.